Enterprise Database Investigation & Architecture Reflection

DBMS Technologies:

- **Oracle**: A relational database management system (RDBMS) developed by Oracle Corporation. First released in 1979, it is one of the world's most widely deployed enterprise database platforms — known for robustness, scalability, and deep support for SQL, PL/SQL, and complex transactional workloads.
- **SQL Server:** A relational database management system developed by Microsoft, first released in 1989. It is one of the most widely used enterprise databases globally, built around T-SQL (Transact-SQL) and tightly integrated with the Microsoft ecosystem — Windows Server, Azure, .NET, and Power BI. It balances enterprise power with relative ease of administration.
- **PostgreSQL**: An open-source, object-relational database management system with over 35 years of active development, originating from the POSTGRES project at UC Berkeley (1986). Known for strict SQL standards compliance, extensibility, and reliability, it has become the world's most popular open-source relational database — favored by developers and enterprises alike as a powerful, cost-free alternative to Oracle and SQL Server.
- **MySQL**: An open-source relational database management system originally developed by MySQL AB, first released in 1995, and now owned by Oracle Corporation. It is the world's most widely deployed open-source database, famous for powering the web — it forms the "M" in the iconic LAMP stack (Linux, Apache, MySQL, PHP). Known for speed, simplicity, and broad platform support, it remains the default database for millions of web applications worldwide.

	Oracle	**SQL Server**	**PostgreSQL**	**MySQL**
Common industries using it	Any sector with high-volume transactions, regulatory requirements, or complex data relationships, e.g. banking & finance, healthcare and telecommunications.	It is especially dominant in mid-size enterprises running Microsoft-stack applications and in sectors that rely on Windows-based infrastructure. e.g. finance, retail, healthcare, and manufacturing.	Its flexibility and zero licensing cost make it the go-to choice for startups, SaaS companies, and any organization prioritizing cost control without sacrificing capability. e.g.technology, fintech, healthcare & e-commerce sectors.	Dominant in web development, e-commerce, media, SaaS, social platforms, and telecom. Powers some of the world's largest websites including Facebook (historically), Twitter, YouTube, and Wikipedia.Extremely popular with startups and small-to-mid businesses building web-facing applications.

Typical company/system environments	Found in large on-premises data centers, enterprise ERP systems (SAP, Oracle E-Business Suite) and core banking platforms.	Common in Windows Server data centers, ERP systems (Microsoft Dynamics, SAP) and CRM platforms.	Common in Linux-based application stacks, cloud-native microservices, web backends (Django, Rails, Node.js), geospatial systems (via PostGIS), data analytics pipelines, and as an Oracle/SQL Server migration target.	The standard database in LAMP/LEMP web stacks, WordPress and CMS platforms, online stores (Magento, WooCommerce), and lightweight SaaS backends.
Enterprise vs open-source usage	A commercial enterprise product — licensed and expensive.	SQL Server is a commercial product, though Microsoft offers a free Express edition for small apps and a Developer edition for non-production use.	PostgreSQL is fully open-source under the PostgreSQL License (similar to MIT). Enterprises that need commercial support and extra features can use distributions like EDB (EnterpriseDB) or Citus.	MySQL is dual-licensed — free under the GPL open-source license, or as a paid MySQL Enterprise Edition with extra security, monitoring, and support tools.

Strengths and trade-offs	Strengths: High availability, ACID compliance and advanced security. Trade-offs: Very high licensing cost, complex administration, and vendor lock-in risk.	Strengths: Excellent tooling (SSMS), strong BI integration (SSIS, SSRS, SSAS), native Azure synergy. Trade-offs: Costly Enterprise edition, historically Windows-centric (Linux support added in 2017), can be heavy for small workloads.	Strengths: Fully free, highly extensible (custom types, functions, extensions), strong ACID compliance. Trade-offs: No official vendor support (unless using EDB), built-in replication is simpler than Oracle RAC, memory tuning can be complex.	Strengths: Very fast for read-heavy workloads, easy to set up, massive community. Trade-offs: Weaker SQL standards compliance than PostgreSQL, limited support for complex queries and advanced data types.
Common tools associated with it	SQL*Plus (CLI), Oracle SQL Developer (GUI IDE), RMAN (backup/recovery), Data Pump (import/export), Oracle Enterprise Manager (monitoring & admin), and GoldenGate (real-time replication).	SSMS (SQL Server Management Studio — the primary GUI), Azure Data Studio (cross-platform IDE), SSIS (ETL/integration), SSRS (reporting), SSAS (analytics/OLAP), SQL Profiler (query tracing), and Database Engine Tuning Advisor.	pgAdmin (primary GUI), psql (CLI), pgBouncer (connection pooling), pg_dump / pg_restore (backup), PostGIS (geospatial extension), Patroni (HA clustering), Flyway / Liquibase (schema migrations), and DBeaver (universal GUI client).	MySQL Workbench (official GUI), phpMyAdmin (web-based GUI, widely used in shared hosting), mysqldump (backup), MySQL Shell (advanced CLI), Percona Toolkit (performance & ops), ProxySQL (connection routing), and DBeaver (universal client).

Integration with cloud or enterprise systems	Available on Oracle Cloud Infrastructure (OCI) as Autonomous Database. Also deployable on AWS (RDS for Oracle), Azure, and Google Cloud.	Natively extends to Azure SQL Database and Azure SQL Managed Instance for fully managed cloud hosting.	Available as managed services on AWS (RDS/Aurora PostgreSQL), Google Cloud SQL / AlloyDB, Azure Database for PostgreSQL, and Supabase.	Available as managed services on AWS (RDS MySQL / Aurora MySQL), Google Cloud SQL, Azure Database for MySQL, and Oracle MySQL HeatWave (cloud-native with in-memory analytics).
Licensing/commercial considerations	Oracle uses per-processor or named-user-plus licensing, which can be extremely costly at scale. There is also an Oracle Standard Edition for smaller setups.	SQL Server comes in several editions: Express (free, limited), Standard (mid-tier), and Enterprise (full features, highest cost). Licensing is either per core or server + CAL (Client Access License). Organizations with existing Microsoft agreements (EA/MSDN) often get favorable pricing.	PostgreSQL is completely free — no licensing fees, no per-core costs, no audit risk. The PostgreSQL License allows use in any commercial product without royalties	The Community Edition is free under GPL — but the GPL requires that applications distributing MySQL also open-source their code, which pushes commercial software vendors toward the paid Commercial License.

Oracle Architecture & Versions Investigation:

Oracle XE — the free taster: XE is Oracle giving you just enough to learn and experiment. It's the full Oracle database engine, but with hard limits bolted on: only 2 CPU threads, 2 GB of RAM, and 12 GB of user data. Once you hit any of those ceilings, the database simply stops using more — it doesn't crash, it just throttles. You get no Oracle support, no Data Guard, no RAC, and you're capped at 3 pluggable databases. It's fine for a developer's laptop or a university project, but the moment you're running a real application with real users, you'll feel those limits fast. The good news: it's genuinely free, forever, with no audit risk.

Oracle SE2 — the affordable but constrained middle ground: Standard Edition 2 is for small-to-mid-size businesses that need a proper, licensed, production Oracle database without the Enterprise price tag. You get unlimited data storage and RAM, Oracle's full support (My Oracle Support), and most of the day-to-day

database features you'd actually use. The catch is the 2 CPU socket ceiling — your server can only have 2 physical CPU sockets, full stop. You also only get 3 pluggable databases, no Data Guard (so no proper standby/disaster recovery), no parallel query processing, and no partitioning. High availability is limited to Oracle's "SE HA" — a simpler, less capable version of RAC. SE2 works well as long as your workload fits within those hardware limits. The problem is there's no upgrade path — the moment your server needs a 3rd CPU socket, you must jump straight to Enterprise Edition, which is a dramatic cost increase with no middle option.

Oracle EE — the full power, full price engine: Enterprise Edition removes every limit. Unlimited CPU, unlimited RAM, unlimited data, unlimited pluggable databases. You also unlock the features that large organizations actually need: Real Application Clusters (RAC) for multi-server high availability, Data Guard for disaster recovery standby databases, parallel query execution for fast analytics, and the full Oracle Enterprise Manager for deep monitoring and administration. Transparent Data Encryption is included. However — and this is critical — many of EE's most valuable features are sold as separate paid add-ons on top of the already-expensive base license. Partitioning, Active Data Guard, Advanced Security, Database Vault, and In-Memory are all extra. A bare EE license is around $47,500 per processor, but a fully loaded production deployment with the common add-ons can realistically cost 100,000–$200,000+ per processor. That's why Oracle EE is mostly found in large banks, telecoms, government agencies, and multinationals where the cost is justified by mission-critical uptime requirements.

	XE	**SE2**	**EE**
Target	Developers / students	small to mid size businesses	Large enterprises
Cost	Free	~$17k/socket	~$47k/processor + add-ons
Real production use	Technically yes, practically no	Yes, within limits	Yes, fully
The main constraint	Hard resource caps	2 CPU socket ceiling	Cost of add-ons

Container Database (CDB):is Oracle's architecture for hosting multiple databases inside a single Oracle instance — introduced in Oracle 12c (2013) as part of the Multitenant feature.

CDB Layers

Oracle instance
Shared memory (SGA) • background processes • one set of redo logs

CDB$ROOT (root container)
System metadata • Oracle-supplied objects • common users & roles

PDB$SEED (read-only template)
Used to quickly provision new PDBs • cannot be modified

Pluggable Databases (PDBs) — each isolated, self-contained

PDB 1 **e.g. HR system** **App schemas** **Local users** **Data files**	**PDB 2** **e.g. Finance** **App schemas** **Local users** **Data files**	**PDB 3** **e.g. Sales** **App schemas** **Local users** **Data files**

Shared storage • control files • redo logs

The Oracle instance at the top is the shared engine — memory, processes, redo logs. Everything below it runs inside that single engine.

The root container (CDB$ROOT) is the brain of the CDB. It holds Oracle's own system metadata, common users, and roles that span all databases. You connect here as a DBA to manage the whole CDB.

The seed (PDB$SEED) is a read-only template. When you want to create a new pluggable database, Oracle copies the seed rather than building from scratch — so provisioning a new database takes seconds instead of hours.

The pluggable databases (PDBs) are the actual databases your applications use. Each one has its own schemas, its own local users, its own data files, and its own application data. They are fully isolated from each other — a crash or problem in PDB1 does not affect PDB2.

Pluggable Database (PDB): is one of the self-contained databases that lives inside a Container Database (CDB). Think of the CDB as an apartment building, and each PDB as a separate apartment — they share the building's infrastructure (electricity, plumbing, security), but each tenant has their own private space, their own lock on the door, and no visibility into their neighbours. A PDB contains everything a traditional standalone Oracle database would have — its own users and roles, its own application schemas (tables, views, indexes, stored procedures), its own tablespaces, and its own physical data files on disk. From an application developer's point of view, a PDB looks and behaves exactly like a regular Oracle database. The application connects to it using a service name and has no idea it's sitting inside a CDB. The name comes from the ability to literally unplug a PDB from one CDB and plug it into another.

Oracle SQL Developer: is Oracle's free,Java-based IDE, official graphical tool for database development and administration. It gives you a visual interface to write SQL, manage database objects, run reports, and administer Oracle databases — without needing to type everything into a command line.

PL/SQL (Procedural Language / SQL): is Oracle's own programming language that extends standard SQL with procedural capabilities — things like variables, conditions, loops, error handling, and reusable code blocks. It lets you write logic that lives and runs directly inside the Oracle database, rather than in your application code.
very PL/SQL program is made of blocks. A block has three sections:

- **DECLARE — optional**

Where you define variables, constants, cursors, and types before using them.

- **BEGIN … END — required**

The executable section — SQL statements, logic, loops, and procedure calls go here.

- **EXCEPTION — optional**

Error handling. Catches runtime errors and lets you respond gracefully instead of crashing.

Oracle Enterprise Architecture: refers to how Oracle database components, services, and infrastructure are structured and connected at an organizational scale — from the physical hardware layer all the way up to the applications that end users interact with. The layers are:
Layer 6 — Client & application tier End-user apps, web apps, APIs, ERP (SAP, Oracle EBS)

↕

Layer 5 — Middleware & integration Oracle SOA Suite, GoldenGate, JDBC/ODBC drivers, REST APIs

↕

Layer 4 — Database logic tier PL/SQL, stored procedures, triggers, packages, views

↕

Layer 3 — CDB / PDB layer Container DB housing multiple Pluggable Databases

↕

Layer 2 — Oracle instance SGA memory, background processes, redo logs

↕

Layer 1 — Physical / storage tier ASM, data files, control files, redo logs, backups (RMAN)

Oracle became the database of choice for large enterprises — banks, telecoms, governments — because it was engineered from the ground up for environments where failure is simply not an option. Its architecture solves three problems that matter most at scale: performance through shared memory and intelligent caching, availability through RAC clustering and Data Guard standby databases, and consolidation through the CDB/PDB model that lets dozens of applications share one engine efficiently. Decades of deployment in critical systems means Oracle is now deeply embedded in enterprise infrastructure — core banking platforms, billing systems, and risk engines are often built on PL/SQL logic that lives inside Oracle itself, making migration a multi-year undertaking rather than a simple switch. That lock-in, combined with aggressive licensing costs, is Oracle's biggest criticism — but for organizations where even minutes of downtime costs millions, the reliability and maturity of the platform continues to justify the price.

Enterprise Decision Thinking

Why might a bank choose Oracle instead of MySQL?

- Banks process millions of transactions daily where data loss or inconsistency is catastrophic — Oracle's ACID compliance, RAC clustering, and Data Guard are built precisely for that
- Oracle offers true high availability with no single point of failure — MySQL simply wasn't designed for that level of redundancy out of the box
- Regulatory requirements in banking demand advanced auditing, encryption, and access controls — Oracle's security stack (TDE, Database Vault, Audit Vault) covers all of it natively
- Core banking systems built decades ago run on Oracle PL/SQL — replacing them is a bigger risk than the licensing cost

Why might a startup choose PostgreSQL instead of Oracle?

- Zero licensing cost means a startup can run a production-grade database from day one without a six-figure Oracle bill
- PostgreSQL runs on any cloud provider as a managed service — AWS RDS, Google Cloud SQL, Supabase — so there's no infrastructure to manage
- Modern frameworks (Django, Rails, Node.js) work naturally with PostgreSQL and the developer community around it is massive
- Startups rarely need RAC or Data Guard at launch — PostgreSQL gives them everything they actually need right now, with room to scale

Why do some companies stay with Microsoft technologies such as SQL Server?

- Organizations already running Windows Server, Active Directory, Azure, and Microsoft 365 get SQL Server integration essentially for free — everything talks to everything
- SSMS is one of the most polished database tools available, and most enterprise developers already know it
- Microsoft's EA licensing agreements often bundle SQL Server at a lower effective cost than buying standalone
- Power BI, SSIS, and SSRS connect natively to SQL Server — the entire BI and reporting stack is already built in

How does cloud infrastructure affect database decisions?

- Cloud providers offer fully managed database services — AWS RDS, Azure SQL, Google Cloud Spanner — which removes the need to hire specialist DBAs for patching, backups, and scaling
- Pay-as-you-go pricing makes it easier to start small and scale, which favors open-source databases like PostgreSQL and MySQL over expensive per-core Oracle licensing
- Oracle responded with Autonomous Database on OCI, and AWS/Azure both support Oracle workloads — so enterprises can lift existing Oracle systems into the cloud without redesigning them
- Cloud-native companies building from scratch almost always choose open-source or managed database services over commercial options — the operational simplicity is too compelling to ignore

Why do enterprise companies sometimes pay for commercial database systems instead of using free alternatives?

- Free software has no SLA — when a critical system goes down at 2am, enterprises need a vendor they can call with a contractual obligation to respond
- Commercial databases like Oracle and SQL Server come with decades of proven reliability in regulated industries where auditors expect certified, supported platforms
- The total cost of running an open-source database at enterprise scale — specialist engineers, third-party HA tools, monitoring, support contracts — often closes the gap with commercial licensing more than people expect
- Risk tolerance is different at enterprise scale — a free database that works 99.9% of the time is unacceptable when that 0.1% represents millions in losses or regulatory penalties

Architecture & Scalability Thinking

Category	Oracle	SQL Server	PostgreSQL	MySQL
Scalability	**Gold standard **24/7 Oracle Support (My Oracle Support). Extensive documentation, patches, and long-term support. Comes with paid licenses.	**Excellent **Microsoft Premier Support with SLAs. Deep integration with Microsoft ecosystem support. Strong partner network.	**Community-led **No official vendor support. Paid support via EDB, 2ndQuadrant, or cloud providers. Large active community is a strong substitute.	**Mixed **Oracle owns MySQL — official support exists but community trust is mixed. Percona offers independent enterprise support.
Cloud readiness	**Strong **Oracle Autonomous DB on OCI. Also available on AWS RDS, Azure. BYOL option. Best performance on Oracle's own OCI cloud.	**Excellent **Azure SQL Database and Managed Instance are first-class. Also on AWS RDS. Azure Hybrid Benefit reduces cloud costs.	**Excellent **Managed on AWS RDS/Aurora, Google Cloud SQL/AlloyDB, Azure, Supabase. The most cloud-portable database available today.	**Very good **Managed on AWS RDS/Aurora, Google Cloud SQL, Azure. Oracle HeatWave adds cloud-native analytics. Widely supported.
Security	**Industry-leading **TDE, Database Vault, Label Security, VPD, Audit Vault, Advanced Security. Full compliance stack for banking and government.	**Very strong **TDE, Always Encrypted, Row-level Security, Dynamic Data Masking, Azure AD integration. Excellent for Microsoft environments	**Good **SSL/TLS, row-level security, role-based access. Solid fundamentals but lacks native enterprise audit and encryption add-ons.	**Moderate **SSL, user privileges, basic encryption. Adequate for web apps. Enterprise security features require MySQL Enterprise add-ons.

Licensing	**Very expensive** ~$47,500/processor for EE. Many critical features sold as costly add-ons. Aggressive audit practices. XE edition is free but limited.	**Expensive **Per-core or CAL licensing. Enterprise edition is costly but more predictable than Oracle. Express edition is free but capped.	**Free **Fully open-source under PostgreSQL License. No per-core costs, no audits, no royalties. Commercial support via EDB available.	**Free / dual **Community edition is free under GPL. Commercial license needed if distributing proprietary software. Enterprise edition is paid.
Enterprise support	**Gold standard **24/7 Oracle Support (My Oracle Support). Extensive documentation, patches, and long-term support. Comes with paid licenses.	**Excellent **Microsoft Premier Support with SLAs. Deep integration with Microsoft ecosystem support. Strong partner network.	**Community-led **No official vendor support. Paid support via EDB, 2ndQuadrant, or cloud providers. Large active community is a strong substitute.	**Mixed **Oracle owns MySQL — official support exists but community trust is mixed. Percona offers independent enterprise support
Performance	**Excellent **In-memory column store, parallel query, advanced optimizer. Best for complex OLTP and mixed workloads at scale.	**Very good **In-memory OLTP, columnstore indexes, and intelligent query processing. Excellent for Windows-based enterprise workloads.	**Very good **MVCC concurrency model prevents read/write blocking. Strong query planner. Excellent for complex analytical queries.	**Good **Very fast for simple read-heavy queries and high-concurrency web apps. Less optimal for complex joins and analytics.
Backup / recovery	**Most advanced **RMAN for full/incremental backups, Flashback for point-in-time recovery, Data Pump for exports. Recovery to any SCN timestamp.	**Very strong **Full, differential, and log backups. Point-in-time restore. Azure Backup integration. Always On provides automatic failover.	**Good **pg_dump, pg_basebackup, WAL archiving for PITR. Tools like Barman and pgBackRest add enterprise-grade backup management.	**Adequate **mysqldump, binary log backups, MySQL Enterprise Backup. Point-in-time recovery available but setup is more manual than Oracle.

Integration	**Oracle ecosystem **Native with Oracle EBS, SAP, GoldenGate, SOA Suite, APEX. JDBC/ODBC for other apps. Strong but proprietary ecosystem.	**Microsoft ecosystem **Native with Power BI, SSIS, SSRS, Active Directory, SharePoint, Dynamics. Best choice if already in Microsoft stack.	**Universal **Works with virtually every language, framework, and tool. First-class support in Django, Rails, Laravel, dbt, Kafka, Airflow.	**Web-native **Built into LAMP/LEMP stack. Native to WordPress, Magento, Joomla. Works with most frameworks. Less strong in enterprise BI tools.

DBMS systems that are commonly used in:

- **Enterprise systems:**

Oracle EE

SQL Server

PostgreSQL

IBM Db2

- **Startups:**

PostgreSQL

MySQL

MongoDB

PlanetScale / Supabase

- **Government sectors:**

Oracle

SQL Server

PostgreSQL

IBM Db2

- **Cloud-native systems:**

PostgreSQL (Aurora / AlloyDB)

MySQL (Aurora)

MongoDB Atlas

CockroachDB / PlanetScale

Google Spanner / DynamoDB

- **Banking systems:**

Oracle EE + RAC

SQL Server

IBM Db2

PostgreSQL (newer banks)

- **Large transactional environments:**

Oracle EE

SQL Server

PostgreSQL

Oracle Exadata

CockroachDB

Migration & Industry Reflection

Companies still choose different DBMS technologies instead of using one universal system because the features being the same on paper does not mean they perform the same under pressure. Every DBMS supports tables and joins, yes — but how they handle ten million concurrent transactions, how they recover from a hardware failure at 3am, how they enforce security at the row level, how they scale across multiple servers, and how much they cost to run at that scale is where the real differences emerge. A bank is not choosing Oracle because it supports triggers — MySQL supports triggers too. It is choosing Oracle because Oracle's RAC architecture keeps the database running even when a server dies, because Data Guard can failover to a standby site in minutes, and because those capabilities have been proven in production banking environments for thirty years. Beyond performance and reliability, the choice is also shaped by ecosystem — a company already running Microsoft 365, Azure, and Windows Server will naturally gravitate toward SQL Server because everything integrates without friction. A startup building on AWS with a Python backend will reach for PostgreSQL because it is free, available as a managed service in two clicks, and works natively with every modern framework. Licensing cost, vendor support contracts, compliance requirements, existing skills on the team, and the technology stack already in place all push organizations in different directions. The features look identical in a textbook. The reality of running those features reliably, securely, and affordably at the scale and risk tolerance of a specific organization is what makes the choice anything but universal.

Why companies migrate from one DBMS to another?

- Licensing costs become unsustainable — Oracle's per-processor fees can reach millions annually, pushing organizations toward PostgreSQL or cloud-managed alternatives
- Cloud adoption forces the conversation — moving infrastructure to AWS or Azure makes managed PostgreSQL or Aurora more attractive than maintaining expensive on-premises Oracle licenses
- The original database was chosen for the wrong reasons — a startup that picked MySQL at launch eventually outgrows it and needs the advanced features of PostgreSQL or Oracle

- Vendor concerns — Oracle's acquisition of MySQL pushed many companies toward MariaDB or PostgreSQL to avoid dependency on a vendor with conflicting interests

What technical and business challenges may appear during migration?

- Schema differences — data types, constraints, and SQL syntax that work in Oracle may not translate directly to PostgreSQL or SQL Server, requiring manual rewriting
- PL/SQL is Oracle-specific — stored procedures, functions, and packages written in PL/SQL must be completely rewritten in the target database's language, which is often the most expensive part of any Oracle migration
- Application code changes — every query, connection string, and database call in the application layer may need updating to match the new database's syntax and drivers
- Data volume — migrating terabytes of production data without data loss, corruption, or unacceptable downtime is an enormous engineering challenge
- Testing coverage — every report, every transaction, every edge case must be retested against the new database to confirm identical behaviour

Why migration projects are considered high-risk in enterprise environments?

- Core systems cannot go down — a bank's payment processing or a telecom's billing system running 24/7 cannot tolerate even minutes of unplanned downtime during cutover
- The scope is always underestimated — what looks like a database swap turns into a full application re-architecture once all the dependencies are discovered
- Data integrity is non-negotiable — any data loss or corruption during migration in a financial or healthcare environment is not just a technical failure but a regulatory and legal one
- Skills gap — migrating from Oracle to PostgreSQL requires expertise in both platforms simultaneously, and finding engineers who know both deeply is genuinely difficult

How database architecture decisions affect long-term software systems?

- The database choice becomes the foundation everything else is built on — application code, reporting tools, integration pipelines, and backup strategies all form around it, making it progressively harder to change
- PL/SQL lock-in is the clearest example — embedding decades of business logic inside Oracle's proprietary language means the organization cannot leave Oracle without rewriting that logic entirely
- Performance assumptions bake in early — applications are designed around how their database handles concurrency, indexing, and transactions, and those assumptions are difficult to reverse later
- Scaling strategy is determined at design time — a system built for vertical scaling on a single Oracle node needs fundamental rearchitecting to scale horizontally later